PRODUCT REQUIREMENTS / API CONTRACT

EnaaConnect

Software Requirements Specification

VERSION	2.1
PREPARED FOR	Projet fil rouge
PREPARED BY	Mehdi El Hajjame
DATE	September 2026
STATUS	MVP specification and implementation baseline

Document control

This SRS consolidates the project requirements in docs/EnaaConnect.md and the behavior currently exposed by the Express application. It uses explicit implementation labels so a planned requirement is never mistaken for a working route.

Field	Value
Document ID	ENAA-SRS-002
Version	2.1
Owner	Mehdi El Hajjame
Baseline	Source reviewed 13 September 2026
Primary source	docs/EnaaConnect.md (version 2.0)
Implementation evidence	server/app.js, route files, controllers, middleware, Mongoose models

Revision history

Version	Date	Change
1.0	July 2026	Initial project description
2.0	September 2026	Resolved role, event, trip, and data-model conflicts
2.1	September 2026	Added diagrams, current API contract, planned endpoint catalog, traceability, and implementation gaps

READING RULE

Implemented means a route is mounted in server/app.js and has a connected controller. Planned means the behavior is required by this SRS but no mounted route was found in the current code review.

Contents

1. Executive summary
2. Product context and scope
3. Stakeholders and roles
4. System overview and architecture
5. Functional requirements
6. Business and authorization rules
7. Data design
8. API documentation conventions
9. Implemented API reference
10. Planned endpoint catalog
11. Non-functional requirements
12. Security requirements
13. Verification strategy
14. Traceability and delivery roadmap
15. Assumptions, gaps, and glossary

DOCUMENT MAP

Sections 1-7 define the product. Sections 8-10 form the API contract. Sections 11-13 define quality and proof. Sections 14-15 connect the requirements to delivery decisions.

1. Executive summary

EnaaConnect is a web platform for the ENAA student community. It centralizes student profiles, a shared feed, events, clubs, conversations, trip planning, notifications, search, reporting, and administration in one authenticated system.

The MVP serves three interaction levels: guests establish access, students participate in the community, and administrators protect platform quality. Club leadership is a responsibility attached to a club record; it is not a third account role.

Product outcomes

- Students find school information and activities in one place.
- Organizers control capacity and participation for events and trips.
- Club leaders manage membership and announcements.
- Administrators approve events and moderate reported content.

Current implementation baseline

Area	Status	Evidence
Authentication	Partial	Register, login, reset request, reset password, current user
Profiles	Implemented core	View public profile, edit profile, upload picture
Feed	Implemented core	List/create/edit/delete posts; like toggle; add/list comments
Events	Implemented core	Create, approve/reject, list approved/pending, join/leave, participants
Clubs, messaging, trips	Planned	Requirements exist; no mounted routes/models found
Notifications, search, reports/admin	Planned	Requirements exist; no mounted routes found

BASELINE COUNT

The current Express application exposes 22 documented routes: 5 authentication, 3 profile, 7 post/comment, and 7 event routes.

2. Product context and scope

2.1 Problem statement

ENAA organizes activities throughout the academic year, but announcements and discussions are spread across several tools. Students can miss information, clubs have limited visibility, and organizers lack a consistent way to manage participation.

2.2 Objectives

- Provide secure ENAA-only account access and recovery.
- Help students present their academic profile and interests.
- Support community posts, questions, images, likes, and comments.
- Create controlled workflows for events, clubs, trips, and messaging.
- Give administrators simple approval and moderation controls.

2.3 MVP scope

Included	Excluded
Authentication and email verification	Payments or online trip booking
Profiles, feed, events, clubs	Public access outside ENAA

Included	Excluded
Private/group messaging and attachments	Video and audio calls
Trips and participant chat	Recommendation algorithms
Notifications, search, reporting, admin	Post search and native mobile applications

2.4 Product boundary

EnaaConnect owns account records, community content, activity membership, notification state, and moderation decisions. External providers may deliver email and store uploaded images or files. The deployed client communicates with the API over HTTPS using JSON or multipart form data.

3. Stakeholders and roles

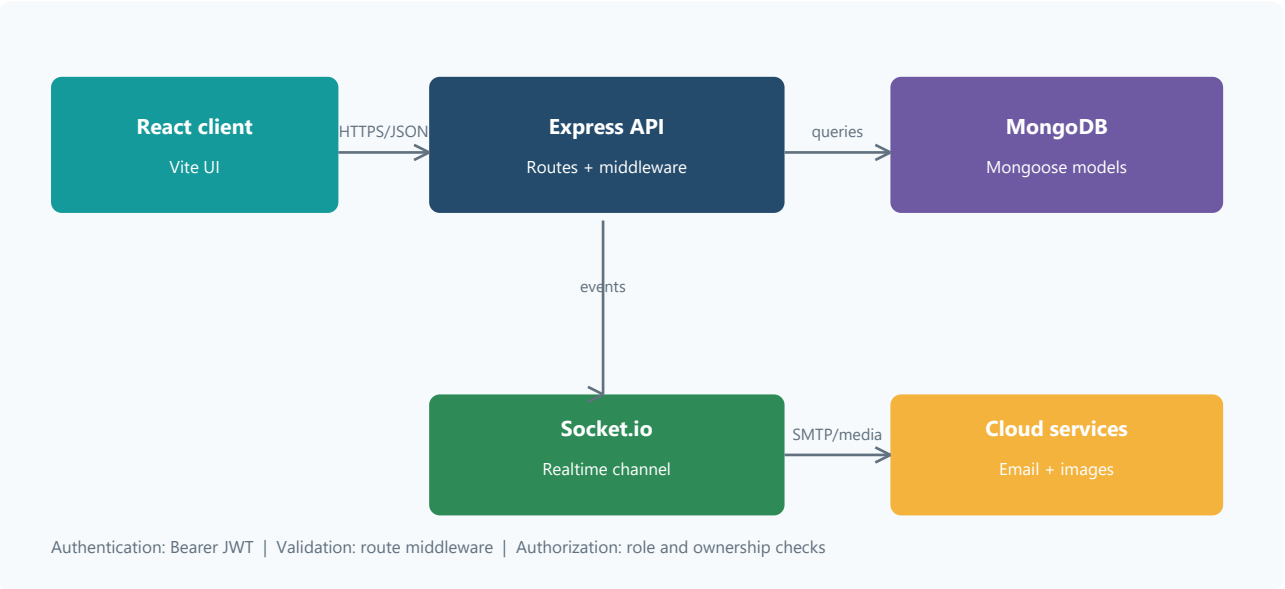
Actor	Responsibilities and permissions
Guest	Register with an @enaa.ma email, log in, and request or complete password recovery.
Student	Manage own profile; use posts; create/join/leave activities; use clubs, messaging, notifications, search, and reports.
Club leader	A student who edits a club they lead, manages its members, and publishes announcements.
Administrator	Manage accounts and clubs, approve/reject events, review reports, and remove inappropriate content.

ROLE MODEL

Only student and admin are stored as User roles. The Club.leader reference grants club leadership for that club.

4. System overview and architecture

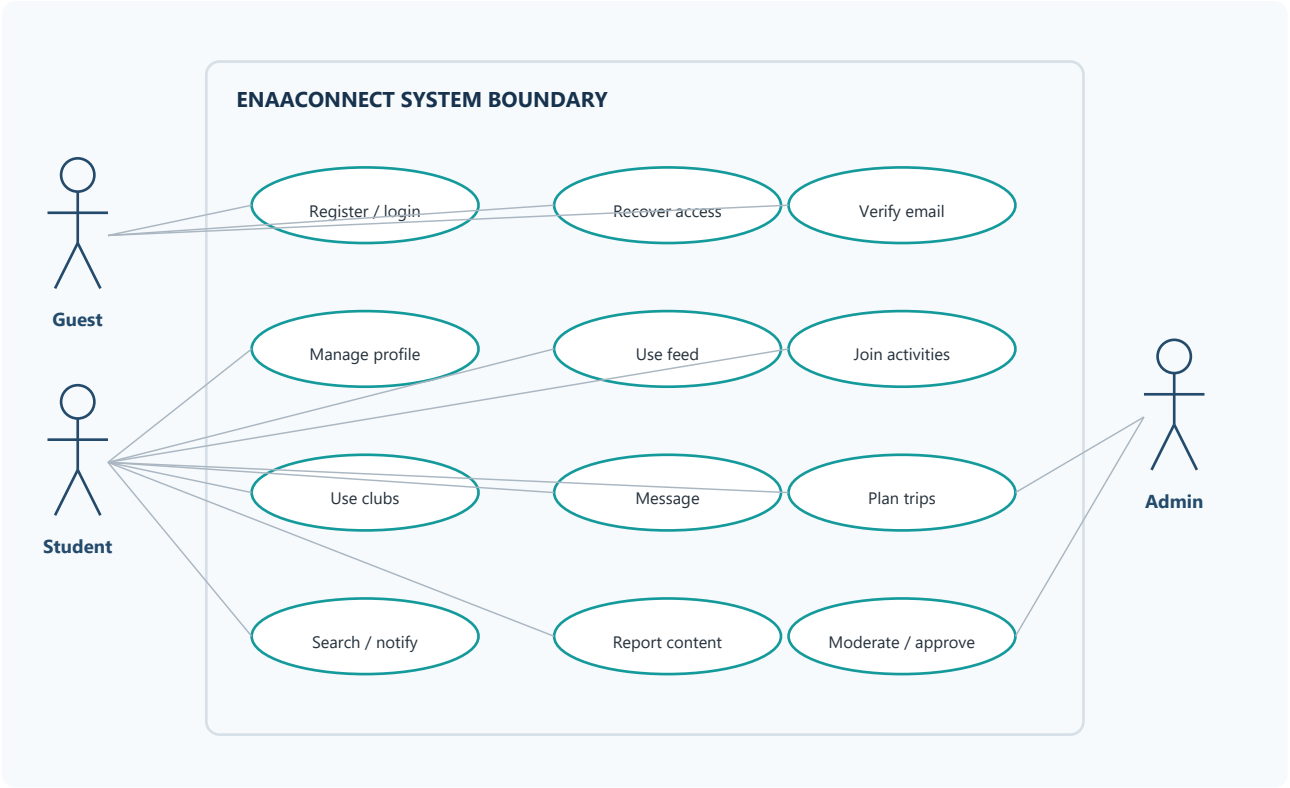
The product follows a client/API/database structure. Requests pass through authentication, route-specific validation, authorization, controller logic, and Mongoose before a standard JSON response is returned.



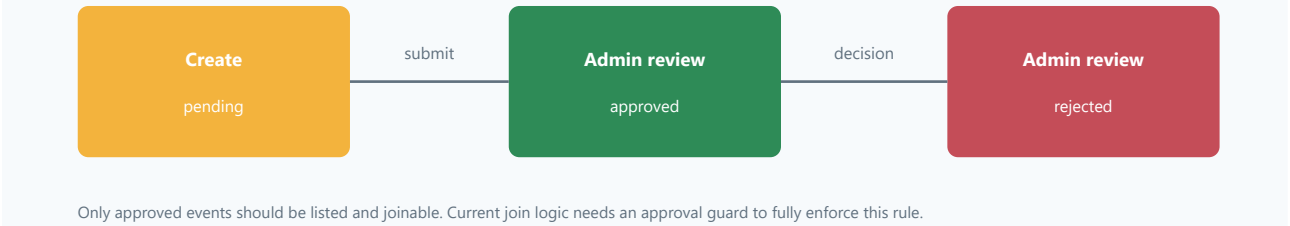
4.1 Request lifecycle

Client action → Express route → Bearer-token authentication when required → body/parameter validation → role or ownership authorization → controller → Mongoose/Cloud service → standard response.

4.2 Use-case view



4.3 Event lifecycle



Only approved events should be listed and joinable. Current join logic needs an approval guard to fully enforce this rule.

5. Functional requirements

5.1 Authentication

- FR-AUTH-01 A guest shall register with full name, ENAA email, password, and password confirmation.
- FR-AUTH-02 Public registration shall always assign the student role.
- FR-AUTH-03 The system shall send a time-limited email verification link.
- FR-AUTH-04 Protected student features shall require a valid authenticated account.
- FR-AUTH-05 A user shall log in with email and password and receive authentication.
- FR-AUTH-06 A user shall log out by removing or invalidating authentication.
- FR-AUTH-07 A guest shall request and complete a time-limited password reset.

5.2 Profiles

- FR-PRO-01 A user shall view another student's safe public profile.
- FR-PRO-02 A user shall edit only their own profile.
- FR-PRO-03 A user shall upload or replace their own profile picture.

- FR-PRO-04 Profile responses shall exclude passwords and security-token fields.

5.3 Student feed

- FR-POST-01 A student shall create a text, question, or image post.
- FR-POST-02 A student shall edit or delete only their own post.
- FR-POST-03 A student shall like or unlike a post once.
- FR-POST-04 A student shall comment on a post.
- FR-POST-05 A club announcement shall be an announcement post linked to its club.
- FR-POST-06 Content and uploads shall be validated before storage.

5.4 Events

- FR-EVT-01 A verified student shall create an event in pending status.
- FR-EVT-02 An administrator shall approve or reject a pending event.
- FR-EVT-03 Only approved events shall be listed, searchable, and joinable.
- FR-EVT-04 The organizer shall edit or cancel their event; material edits return it to pending.
- FR-EVT-05 Participation shall prevent duplicates and capacity overflow.
- FR-EVT-06 Participants shall leave an event.
- FR-EVT-07 Students shall comment on approved events.
- FR-EVT-08 The interface shall provide a shareable event link.

5.5 Clubs

- FR-CLUB-01 A verified student shall create a club.
- FR-CLUB-02 The creator becomes leader and first member.
- FR-CLUB-03 Students shall join or leave clubs.
- FR-CLUB-04 Only the club leader edits the club or manages members.
- FR-CLUB-05 The leader shall publish club announcement posts.

5.6 Messaging

- FR-MSG-01 A student shall create or reuse a direct conversation.
- FR-MSG-02 A student shall create a group conversation.
- FR-MSG-03 Only participants view or send conversation messages.
- FR-MSG-04 Messages reference one conversation and may carry validated attachments.
- FR-MSG-05 Participants shall mark messages as read.

5.7 Trips

- FR-TRIP-01 A verified student shall create and manage a separate trip.
- FR-TRIP-02 Participation shall prevent duplicates and capacity overflow.
- FR-TRIP-03 Participants shall use the trip conversation.
- FR-TRIP-04 The MVP shall not process trip payments.

5.8 Notifications and search

- FR-NOT-01 Defined social, activity, message, and moderation actions shall create notifications.
- FR-NOT-02 Users shall list and mark notifications read.
- FR-SRCH-01 Students shall search users by name, study field, or interest.
- FR-SRCH-02 Students shall search clubs by name and approved events by title, category, date, or location.
- FR-SRCH-03 Search shall not expose private fields.

5.9 Reporting and administration

- FR-ADM-01 Students shall report supported content with a reason.
- FR-ADM-02 Reports shall use pending, resolved, or dismissed status.
- FR-ADM-03 Administrators shall review reports and remove inappropriate content when needed.
- FR-ADM-04 Administrator endpoints shall enforce the admin role on the server.

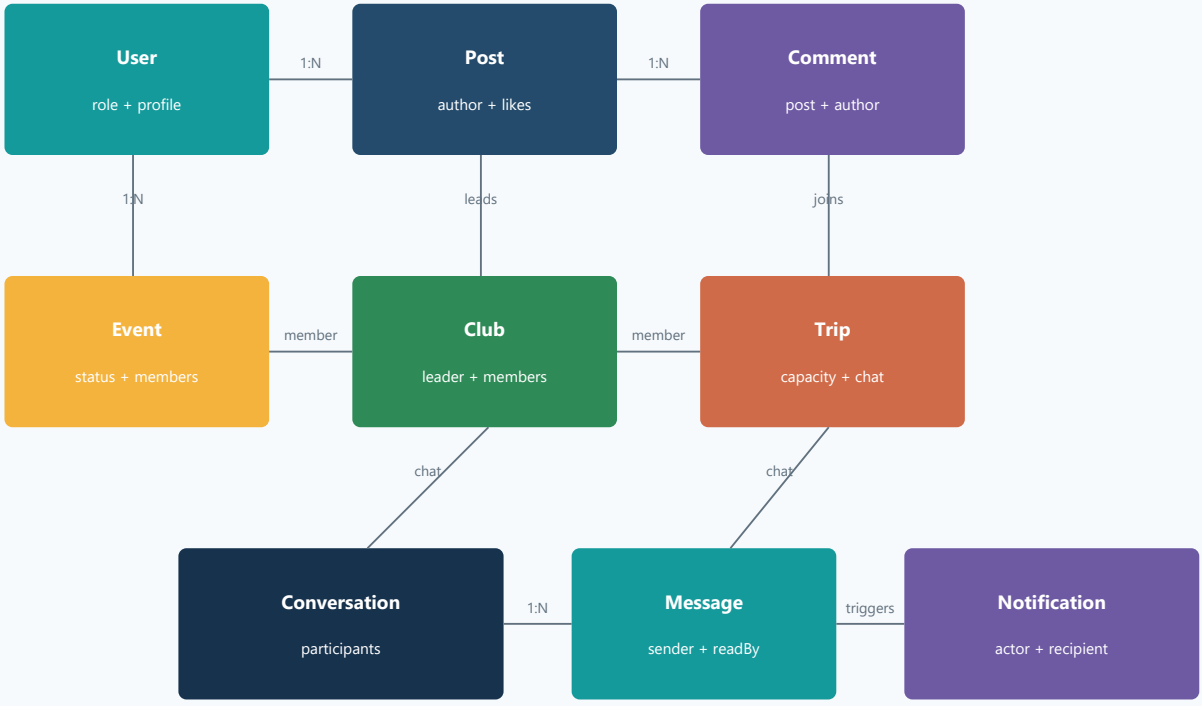
6. Business and authorization rules

ID	Rule
BR-01	Public registration always assigns student.
BR-02	Club leadership comes from Club.leader.
BR-03	The server checks ownership; the interface is not an authorization boundary.
BR-04	A user cannot join an event, club, or trip twice.
BR-05	Event and trip capacity cannot be exceeded.
BR-06	A club creator becomes leader and first member.
BR-07	Only approved events are visible in public results and joinable.
BR-08	Only conversation participants access messages and attachments.
BR-09	Admin endpoints return HTTP 403 for authenticated non-admin users.

Authorization matrix

Operation	Guest	Student	Club leader	Admin
Register / login / recovery	Yes	Yes	Yes	Yes
View/edit own profile	No	Yes	Yes	Yes
Create posts and events	No	Yes	Yes	Policy choice
Manage a club	No	No	Own club	Platform-wide
Approve events / resolve reports	No	No	No	Yes

7. Data design



Solid entities exist or are specified in the SRS. Current code implements User, Post, Comment, and Event models.

Collection	Key fields	Status
Users	fullName, email, password, role, profile, verification/reset fields	Implemented
Posts	author, content, images, likes, comments	Implemented
Comments	post, author, content	Implemented
Events	title, date/time, category, organizer, participants, status/review	Implemented
Clubs	name, description, logo, leader, members	Planned
Conversations / Messages	participants; sender, content, attachments, readBy	Planned
Trips	organizer, conversation, destination, budget, capacity, participants	Planned
Notifications	recipient, actor, type, related entity, isRead	Planned
Reports	reporter, target, reason, status, reviewer	Planned

7.1 Data integrity

- MongoDB references use ObjectId values and route parameters must validate the 24-character hexadecimal format.
- User arrays for likes and membership must reject duplicates in controller logic.
- Capacity checks must happen immediately before persistence; later hardening may use an atomic update to prevent race conditions.
- Sensitive fields use select:false and must also be omitted from response DTOs.

8. API documentation conventions

8.1 Base and transport

Development base: http://localhost:<PORT>. API routes begin with /api. Production uses HTTPS. JSON is used except image upload routes, which use multipart/form-data.

8.2 Authentication

Protected routes require Authorization: Bearer <jwt>. The current API returns the JWT inside the successful register, login, and reset-password response data.

8.3 Standard response envelope

Successful response

```
{ "success": true, "message": "Operation completed", "data": { } }
```

Error response

```
{ "success": false, "message": "Clear error message" }
```

8.4 Common status codes

Code	Meaning
200	Successful read, update, join/leave, or deletion
201	Resource created
400	Invalid body, parameter, upload, or token state
401	Missing or invalid authentication
403	Authenticated but lacks the required role/ownership
404	Resource or route not found
409	Duplicate/conflicting state or capacity reached
500	Unexpected server or provider failure

8.5 Implementation labels

Label	Meaning
Implemented	Mounted route and connected controller found
Partial	Route works but does not yet meet every SRS rule
Planned	Required contract proposed for later implementation

9. Implemented API reference

The following routes are mounted in the current Express application. The status column measures alignment with the SRS, not merely whether code exists.

Area	Method	Route	Access	SRS alignment
Authentication	POST	/api/auth/register	Public	Partial
Authentication	POST	/api/auth/login	Public	Implemented
Authentication	POST	/api/auth/forget-password	Public	Partial
Authentication	PATCH	/api/auth/reset-password/:token	Public	Partial

Area	Method	Route	Access	SRS alignment
Authentication	GET	/api/auth/me	Bearer JWT	Implemented
Profiles	PATCH	/api/users/profile-picture	Bearer JWT	Implemented
Profiles	GET	/api/users/:userId	Bearer JWT	Implemented
Profiles	PATCH	/api/users/update-profile	Bearer JWT	Implemented
Posts	GET	/api/posts	Bearer JWT	Implemented
Posts	POST	/api/posts	Student JWT	Implemented
Posts	PATCH	/api/posts/:postId	Student owner JWT	Implemented
Posts	DELETE	/api/posts/:postId	Student owner JWT	Implemented
Posts	PATCH	/api/posts/:postId/like	Student JWT	Implemented
Comments	POST	/api/posts/:postId/comments	Student JWT	Implemented
Comments	GET	/api/posts/:postId/comments	Bearer JWT	Implemented
Events	POST	/api/events	Student JWT	Implemented
Events	GET	/api/events	Bearer JWT	Implemented
Events	GET	/api/events/pending	Admin JWT	Implemented
Events	GET	/api/events/:eventId/participants	Student JWT	Implemented
Events	POST	/api/events/:eventId/join	Student JWT	Partial
Events	DELETE	/api/events/:eventId/join	Student JWT	Implemented
Events	PATCH	/api/events/:eventId/review	Admin JWT	Implemented

9.1 Authentication

POST	/api/auth/register	PARTIAL
Access	Public	
Request	JSON: fullName, email, password, confirmPassword	
Success	201 account + token	
Important failures	400 invalid; 409 duplicate; 500	

POST	/api/auth/login	IMPLEMENTED
Access	Public	
Request	JSON: email, password	
Success	200 user + token	
Important failures	400 missing; 401 bad credentials; 500	

POST	/api/auth/forget-password	PARTIAL
Access	Public	
Request	JSON: email	
Success	200 empty data; reset email	
Important failures	404 unknown email; 500 delivery/server	

PATCH	/api/auth/reset-password/:token	PARTIAL
Access	Public	
Request	JSON: password, confirmPassword	
Success	200 token	
Important failures	400 invalid/expired; 500	

GET	/api/auth/me	IMPLEMENTED
Access	Bearer JWT	
Request	No body	
Success	200 safe current-user profile	
Important failures	401 invalid/missing user/token; 500	

9.2 Profiles

PATCH	/api/users/profile-picture	IMPLEMENTED
Access	Bearer JWT	
Request	multipart: profilePicture (1 JPEG/PNG/WebP, max 5 MB)	
Success	200 updated profile	
Important failures	400 file/type/size/body; 401; 500	

GET	/api/users/:userId	IMPLEMENTED
Access	Bearer JWT	
Request	24-char userId path	
Success	200 safe public profile	
Important failures	400 ID; 401; 404 student	

PATCH	/api/users/update-profile	IMPLEMENTED
Access	Bearer JWT	
Request	Any allowed JSON fields: fullName, fieldOfStudy, academicYear, biography, interests	
Success	200 updated profile	
Important failures	400 unknown/invalid/empty; 401; 404; 500	

9.3 Posts

GET	/api/posts	IMPLEMENTED
Access	Bearer JWT	
Request	No body	
Success	200 newest-first post list	
Important failures	401; 500	

POST	/api/posts	IMPLEMENTED
Access	Student JWT	
Request	multipart: content + up to 5 images (5 MB each)	
Success	201 post	
Important failures	400 content/upload; 401; 403; 500	

PATCH	/api/posts/:postId	IMPLEMENTED
Access	Student owner JWT	
Request	JSON: content	
Success	200 updated post	
Important failures	400 ID/body; 401; 403 owner/role; 404; 500	

DELETE	/api/posts/:postId	IMPLEMENTED
Access	Student owner JWT	
Request	No body	
Success	200 deleted post id	
Important failures	400 ID; 401; 403; 404; 500	

PATCH	/api/posts/:postId/like	IMPLEMENTED
Access	Student JWT	
Request	No body; toggles current user's like	
Success	200 liked flag + count	
Important failures	400 ID; 401; 403; 404; 500	

9.4 Comments

POST	/api/posts/:postId/comments	IMPLEMENTED
Access	Student JWT	
Request	JSON: content (1-1000 chars)	
Success	201 comment + count	
Important failures	400 ID/body; 401; 403; 404; 500	

GET	/api/posts/:postId/comments	IMPLEMENTED
Access	Bearer JWT	
Request	24-char postId path	
Success	200 oldest-first comments + count	
Important failures	400 ID; 401; 404; 500	

9.5 Events

POST	/api/events	IMPLEMENTED
Access	Student JWT	
Request	JSON: title, description, date, time, location, category, maximumParticipants	
Success	201 pending event	
Important failures	400 body/date; 401; 403; 500	

GET	/api/events	IMPLEMENTED
Access	Bearer JWT	
Request	No body	
Success	200 approved events, chronological	
Important failures	401; 500	

GET	/api/events/pending	IMPLEMENTED
Access	Admin JWT	
Request	No body	
Success	200 pending events, oldest first	
Important failures	401; 403; 500	

GET	/api/events/:eventId/participants	IMPLEMENTED
Access	Student JWT	
Request	24-char eventId path	
Success	200 approved-event participant list	
Important failures	400 ID; 401; 403; 404; 500	

POST	/api/events/:eventId/join	PARTIAL
Access	Student JWT	
Request	24-char eventId path	
Success	200 membership/capacity state	
Important failures	400 ID; 401; 403; 404; 409 duplicate/full; 500	

DELETE	/api/events/:eventId/join	IMPLEMENTED
Access	Student JWT	
Request	24-char eventId path	
Success	200 membership/capacity state	
Important failures	400 ID; 401; 403; 404; 409 not joined; 500	

PATCH	/api/events/:eventId/review	IMPLEMENTED
Access	Admin JWT	
Request	JSON: status = approved | rejected	
Success	200 review record	
Important failures	400 ID/body; 401; 403; 404; 409 not pending; 500	

9.6 Current request examples

Register

```
POST /api/auth/register Content-Type: application/json { "fullName": "Student Name", "email": "student@enaa.ma", "password": "strongPass123", "confirmPassword": "strongPass123" }
```

Create event

```
POST /api/events Authorization: Bearer <jwt> Content-Type: application/json { "title": "Design Workshop", "description": "Hands-on interface design session", "date": "2026-10-10", "time": "14:30", "location": "ENAA Lab 2", "category": "workshops", "maximumParticipants": 30 }
```

10. Planned endpoint catalog

These endpoints complete the SRS feature set. They are proposed contracts and are not currently mounted. Before implementation, keep route naming consistent with the existing API and add strict validation before each controller.

Area	Method	Proposed route	Purpose / main input
Auth	GET	/api/auth/verify-email/:token	Verify an email-token and activate account
Auth	POST	/api/auth/logout	End client/server authentication session
Events	GET	/api/events/:eventId	Read one approved event
Events	PATCH	/api/events/:eventId	Organizer edits allowed fields; material edits return pending
Events	DELETE	/api/events/:eventId	Organizer cancels own event
Events	POST	/api/events/:eventId/comments	Add comment to approved event
Events	GET	/api/events/:eventId/comments	List event comments
Clubs	POST	/api/clubs	Create club; creator becomes leader/member
Clubs	GET	/api/clubs	List/search visible clubs
Clubs	GET	/api/clubs/:clubId	Read club details
Clubs	PATCH	/api/clubs/:clubId	Leader edits club
Clubs	DELETE	/api/clubs/:clubId	Leader/admin deletes club
Clubs	POST	/api/clubs/:clubId/join	Join club once
Clubs	DELETE	/api/clubs/:clubId/join	Leave club; leader rule required
Clubs	GET	/api/clubs/:clubId/members	List members
Clubs	POST	/api/clubs/:clubId/members/:userId	Leader adds member
Clubs	DELETE	/api/clubs/:clubId/members/:userId	Leader removes member
Clubs	POST	/api/clubs/:clubId/announcements	Leader creates announcement post
Conversations	POST	/api/conversations/direct	Create/reuse direct conversation: participantId
Conversations	POST	/api/conversations/group	Create group: groupName, participantIds
Conversations	GET	/api/conversations	List current user's conversations

Area	Method	Proposed route	Purpose / main input
Conversations	GET	/api/conversations/:conversationId/messages	Paginated participant-only messages
Messages	POST	/api/conversations/:conversationId/messages	Send content and validated attachments
Messages	PATCH	/api/conversations/:conversationId/read	Mark visible messages read
Trips	POST	/api/trips	Create trip and participant conversation
Trips	GET	/api/trips	List trips with filters/pagination
Trips	GET	/api/trips/:tripId	Read trip details
Trips	PATCH	/api/trips/:tripId	Organizer edits trip
Trips	DELETE	/api/trips/:tripId	Organizer cancels trip
Trips	POST	/api/trips/:tripId/join	Join once if capacity exists
Trips	DELETE	/api/trips/:tripId/join	Leave trip
Trips	GET	/api/trips/:tripId/conversation	Participant opens trip chat
Notifications	GET	/api/notifications	List current user's notifications
Notifications	PATCH	/api/notifications/:notificationId/read	Mark one notification read
Notifications	PATCH	/api/notifications/read-all	Mark all current-user notifications read
Search	GET	/api/search/users?q=&field;=&interest;=	Search safe student fields
Search	GET	/api/search/clubs?q=	Search clubs by name
Search	GET	/api/search/events?q=&category;=&date;=&location;=	Search approved events
Reports	POST	/api/reports	Create report: targetType, targetId, reason
Admin	GET	/api/admin/reports	List/filter reports
Admin	PATCH	/api/admin/reports/:reportId	Resolve or dismiss report
Admin	GET	/api/admin/users	List/search users
Admin	PATCH	/api/admin/users/:userId/status	Suspend or reactivate account
Admin	DELETE	/api/admin/content/:type/:id	Remove inappropriate content
Admin	GET	/api/admin/clubs	Review/manage clubs

10.1 Rules for every new body-accepting route

- Authenticate first when the operation is protected.
- Validate the complete body, reject unknown fields, normalize strings, and validate ObjectIds.
- Apply role, ownership, club-leader, or conversation-membership authorization before changing data.
- Return the standard success/error envelope and document normal and failure status codes.
- Paginate collection endpoints for posts, events, messages, trips, notifications, reports, and admin lists.

11. Non-functional requirements

Category	ID	Requirement
Performance	NFR-PERF-01	95% of normal API requests complete within 2 seconds, excluding large uploads and email delivery.
Performance	NFR-PERF-02	Large lists use pagination; frequently filtered fields use indexes.
Reliability	NFR-REL-01	Create, join, leave, approval, and moderation return consistent results.
Reliability	NFR-REL-02	Failed email delivery must not leave misleading verification/reset state.
Reliability	NFR-REL-03	Failures are logged without passwords or tokens.

Category	ID	Requirement
Usability	NFR-USE-01	Responsive interface for desktop and mobile browser sizes.
Usability	NFR-USE-02	Forms show clear validation and failure messages.
Accessibility	NFR-A11Y-01	Main interactions are keyboard accessible with visible focus.
Compatibility	NFR-COMP-01	Support current Chrome, Edge, and Firefox; production API uses HTTPS JSON.

11.1 Operational targets

- Health: the root endpoint confirms the API process is reachable; production should add database readiness.
- Observability: log request failures and provider errors with a request identifier.
- Backups: database backup and restore procedures must be defined before production use.
- Configuration: JWT, MongoDB, email, Cloudinary, allowed origin, and port values live in environment variables.

12. Security requirements

ID	Requirement	Current note
SEC-01	Hash passwords with bcrypt; never return plain passwords.	Implemented in model
SEC-02	Store JWT, DB, email, and media secrets in environment variables.	Used by current code
SEC-03	Hash and expire reset/verification tokens; never log them.	Reset token implemented; verification route pending
SEC-04	Verify JWT and confirm user still exists.	Implemented
SEC-05	Enforce role, ownership, leadership, and conversation membership.	Role/owner partial; later areas pending
SEC-06	Validate every body before controllers and reject unknown fields.	Profiles/posts/events strong; auth partial
SEC-07	Allowlist upload types and size limits.	JPEG/PNG/WebP, 5 MB implemented
SEC-08	Exclude password and token hashes from responses.	Implemented response DTOs
SEC-09	Use HTTPS and explicit CORS allowed origins in production.	Deployment requirement
SEC-10	Rate-limit registration, login, and password reset.	Planned

SECURITY GAP

Current registration returns a JWT immediately and the protection middleware does not check isVerified. This does not yet enforce FR-AUTH-03/04. Add email verification before claiming the full authentication requirement is complete.

EVENT GAP

The current join-event controller checks existence, duplicates, and capacity but does not check status === approved. Add the approval check before the endpoint can satisfy BR-07.

13. Verification strategy

Each important behavior needs a normal case, an edge case, and a failure or authorization case. API tests should assert both the HTTP response and the database result.

Area	Normal evidence	Failure evidence
Authentication	Valid ENAA account registers/logs in and returns safe user data	Unknown fields, bad email/password, expired reset token, and unverified access fail
Profiles	Owner updates allowed fields; valid picture is saved	Unknown field, bad ObjectId, oversized/wrong upload fail

Area	Normal evidence	Failure evidence
Posts	Create/list/edit/delete/like/comment update counts correctly	Non-owner edit/delete, invalid content, missing post fail
Events	Pending creation, admin review, approved listing, capacity state	Non-admin review, duplicate join, full event, unapproved join fail
Planned modules	One end-to-end test per endpoint group plus membership/ownership cases	Cross-user access and invalid state transitions fail

13.1 Example verification sequence

```
1. POST /api/auth/register with an @enaa.ma account; expect 201. 2. POST /api/auth/login; expect 200 and copy data.token.
3. POST /api/events with Authorization: Bearer <token>; expect 201 and status pending. 4. GET /api/events with the student
token; the pending event must not appear. 5. PATCH /api/events/:id/review as admin with {"status":"approved"}; expect 200.
6. GET /api/events; the event now appears. 7. POST /api/events/:id/join; expect 200 and participantsCount 1. 8. Repeat
join; expect 409.
```

13.2 Definition of done

- Observable normal behavior works.
- Important validation and authorization failures work.
- Sensitive data is absent from API responses and logs.
- Database state matches the response.
- Relevant tests pass and the SRS/API documentation matches the implemented behavior.

14. Traceability and delivery roadmap

14.1 Requirement-to-route traceability

Requirement group	Implemented evidence	Remaining delivery
FR-AUTH	/api/auth register/login/recovery/me	Email verification, logout strategy, strict auth-body validation, verified-user gate
FR-PRO	/api/users profile routes	Search integration and optional account settings
FR-POST	/api/posts and nested comments	Question/announcement types, notifications, pagination
FR-EVT	/api/events core routes	Read detail, edit/cancel, event comments/share, approved-join guard
FR-CLUB	None	Model, validation, routes, leader/membership/announcements
FR-MSG	Socket dependency only	Conversation/message models, REST history, Socket authorization, attachments/read state
FR-TRIP	None	Trip model, membership, capacity, linked conversation
FR-NOT/SRCH/ADM	Event admin review only	Notifications, search, reports, admin users/clubs/content

14.2 Recommended MVP order

Priority	Delivery slice	Reason
1	Close authentication/security gaps	All protected features depend on trustworthy access
2	Finish event rules and event detail/comments	Current work is close to complete and visible
3	Clubs and announcements	Core community requirement and reuses posts/membership
4	Notifications and search	Connects existing posts/events/clubs
5	Conversations and messages	Largest realtime/authorization slice
6	Trips and participant chat	Reuses capacity and conversation rules

Priority	Delivery slice	Reason
7	Reports and remaining admin	Completes moderation before release

DEADLINE FOCUS

Finish required and broken behavior before optional architecture or visual polish. Keep each slice model → validation → authorization → controller → route → test → interface.

15. Assumptions, gaps, and glossary

15.1 Assumptions

- All registered users must use an @enaa.ma email.
- A verified student can create an event; administrators only review it.
- Trips remain separate from events despite the current Event model containing a trips category.
- Logout may be client-side token deletion for the MVP unless server-side revocation is later required.

15.2 Known specification/implementation gaps

Gap	Decision needed or correction
Email verification not connected	Add verification endpoint, email dispatch, and verified-user middleware
Auth body validation occurs in controllers	Move complete strict schemas to route middleware
Trip is both an event category and a separate feature	Remove trips from event categories when Trip is implemented
Join-event approval guard missing	Reject non-approved events before duplicate/capacity checks
No pagination on current list routes	Add simple page/limit query validation before larger data volumes
No test script	Add targeted API integration tests for implemented core routes

15.3 Glossary

Term	Meaning
JWT	Signed token used to authenticate API requests
DTO	Explicit safe response object returned instead of the raw database record
MVP	Minimum product scope required for a useful first release
ObjectId	MongoDB identifier represented as 24 hexadecimal characters
Ownership	The authenticated user created or controls a resource
Participant	A user included in an event, trip, club, or conversation
SRS	Software Requirements Specification

END OF SPECIFICATION

This document is the baseline for implementation and review. Update endpoint status and traceability whenever a route is added, changed, or removed.